Neural Network Day 3

역전파 (backpropagation)

HYUNDAI AI Insight Campus · 온디바이스 AI · Neural Network 3/5

2026. 10. 22 (목) 09:00–18:00 · 8교시 · Google Colab

계산 그래프 | 연쇄 법칙 | ReLU · Sigmoid | Affine · Softmax | Gradient check | MNIST MLP

강사 : 김정인 · 김구수

오늘의 학습 목표

수업이 끝나면 여러분은 다음을 할 수 있습니다

1 수치 미분의 한계

파라미터 수로 비용을 계산한다 — 79,510 개 → step 당 손실 159,020 번

2 계산 그래프

사과 · 귤 쇼핑 그래프의 순전파 값과 역전파 기울기를 손으로 채운다

3 연쇄 법칙

덧셈 노드 · 곱셈 노드의 backward 규칙으로 MulLayer · AddLayer 를 만든다

4 계층 구현

Relu · Sigmoid · Affine · SoftmaxWithLoss 의 forward / backward 를 구현한다

5 Gradient check

상대 오차로 역전파를 수치 미분과 비교하고 , 숨은 버그를 찾는다

6 미니배치 학습

미니배치 SGD 로 나선 데이터와 MNIST(test ≥ 0.95) 를 학습시킨다

시간표

8 교시 × 50 분 · 점심 12:00–13:00 · 오전 원리 (계산 그래프 · 연쇄 법칙), 오후 계층 구현 → 검증 → 학습

교시	시간	주제	형태	애니메이션
1	09:00–09:50	Day 2 복습 퀴즈 · 수치 미분의 한계	퀴즈 + 강의	D3-1 수치 미분 vs 역전파
2	10:00–10:50	계산 그래프 — 사과 · 귤 쇼핑 예제	강의 + 실습	D3-2 계산 그래프
3	11:00–11:50	연쇄 법칙 — 덧셈 · 곱셈 노드 , MulLayer · AddLayer	강의 + 실습	D3-3 연쇄 법칙
4	13:00–13:50	활성화 계층 — Relu · Sigmoid forward / backward	강의 + 실습	D3-4 ReLU · Sigmoid 계층
5	14:00–14:50	Affine · Softmax-with-Loss 역전파	강의 + 실습	D3-5 shape 퍼즐 · D3-6 y − t
6	15:00–15:50	Gradient check — TwoLayerNet · 상대 오차 · 버그 찾기	강의 + 실습	D3-7 Gradient check
7	16:00–16:50	미니배치 SGD — 나선 3 클래스 분류 · 결정 경계	강의 + 실습	D3-8 미니배치 SGD
8	17:00–17:50	미니 프로젝트 — MNIST 2 층 MLP (test ≥ 0.95)	개인 실습	D3-9 MNIST 2 층 MLP

오늘의 자료 · 온라인 링크

예측 (애니메이션) → 검증 (노트북) → 점검 (워크시트) — 모두 브라우저에서 바로 열립니다

기본 주소 (Day 3)

https://imguru-mooc.github.io/AI_CAMPUS/NN-day3/

animation.html	애니메이션 9 개 (D3-1 ~ D3-9) — #d3-5 처럼 바로가기
worksheet.html	개념 워크시트 — #t1 ~ #t8 교시 바로가기
NN_Day3_lecture.ipynb	강의 노트북 — 1~8 교시 문제 + MNIST 미니 프로젝트 (Colab 에서 열기)
NN_Day3.pdf	오늘 슬라이드 PDF
Day 2 복습 퀴즈	09:00 시작 — 강사가 Google Forms 링크 공유 (20 문항 · 15 분)

노트북은 " 파일 → Drive 에 사본 저장 " 후 작업 · 런타임이 끊기면 환경 셀 (계층 클래스 포함) 부터 다시 실행

오늘 만드는 것 — 학습의 고리 중 " 기울기 " 칸

Day 2: 순전파 · 손실을 완성했지만 기울기는 손 유도 · 수치 미분 . 오늘은 역방향으로 한 번에 모든 기울기를 구합니다

순전파

Y = XW + b → 활성화
Day 2

→

손실

softmax · CE
Day 2

→

역전파

dout 을 거꾸로 전달
Day 3 (오늘)

→

갱신

W ← W − η·dW
Day 3 미니배치 SGD

수치 미분 (Day 1 · 2)

```
P = 784·100 + 100 + 100·10 + 10 = 79,510
손실 계산 2P = 159,020 번 / step
≈ 69 초 / step → 1 epoch ≈ 11.5 시간
```

파라미터마다 손실 2 번 , 순전파 한 번도 P 에 비례
→ 전체 비용 ∝ P^2

역전파 (Day 3)

```
순전파 1 번 + 역방향 1 번 (≈ 순전파 2~3 배 )
≈ 2 ms / step → 1 epoch ≈ 1.2 초
약 35,000 배 빠르다
```

측정 : 서버 CPU · 배치 100 기준
Colab 은 기기마다 달라 " 수십 초 vs 수 ms" 로 기억

1 Day 2 복습 퀴즈 · 수치 미분의 한계

09:00–09:50

학습 목표

- 09:00–09:15 Day 2 복습 퀴즈 (20 문항)
- 수치 미분 : 파라미터마다 손실 2 번 → step 당 2P 번
- 순전파 한 번도 P 에 비례 → 전체 ∝ P^2
- 역전파 = 순전파 1 + 역방향 1 (≈ 순전파 2~3 배)

노트북 실습

1. 문제 1-1 count_params(net)
2. 문제 1-2 손실 계산 횟수 2 * P
3. 문제 1-3 시간 측정 · 784-100-10 외삽

▶ 애니메이션 D3-1 수치 미분 vs 역전파 비용

핵심 개념

파라미터 수

```
P = 784·100 + 100 + 100·10 + 10
  = 79,510
```

손실 계산

```
2 * P = 159,020 번 / step
```

작은 망

```
784-1-10: 805 · 784-2-10: 1,600
784-4-10: 3,190  → 시간 재고 외삽
```

측정

```
손실 1 회 ≈ 0.43 ms → 수치 미분 ≈ 69 s
역전파 ≈ 2 ms (약 35,000 배)
```

체크포인트

1 epoch(600 step) 기준 수치 미분 ≈ 11.5 시간, 역전파 ≈ 1.2 초. Colab 숫자는 기기마다 다르니 비율로 기억.

2 계산 그래프

10:00–10:50

학습 목표

- 노드 = 연산 , 엣지 = 값
- 국소 계산 : 노드는 자기 입력 · 출력만 안다
- 순방향 = 값 계산 , 역방향 = 기울기 계산
- 사과 · 귤 쇼핑 — 소비세 1.1

노트북 실습

1. 문제 2-1 순전파 값 채우기
2. 문제 2-2 손으로 역전파 값 채우기
3. 문제 2-3 수치 미분으로 확인

▶ 애니메이션 D3-2 계산 그래프 — 사과 · 귤 쇼핑

핵심 개념

순전파

```
사과 100 × 2 = 200,   귤 150 × 3 = 450
(200 + 450) × 1.1 = 715
```

역전파 (dout = 1)

```
d 소비세 = 650,   d 합계 = 1.1
d 사과가격 = 2.2,   d 사과개수 = 110
d 귤가격 = 3.3,   d 귤개수 = 165
```

의미

```
사과 가격 1 원 ↑ → 최종 금액 2.2 원 ↑
```

체크포인트

역방향 한 번으로 모든 입력 (가격 · 개수 · 소비세) 의 기울기가 동시에 나온다 — 수치 미분은 입력마다 따로 .

3 연쇄 법칙

11:00–11:50

학습 목표

- 합성 함수의 미분 = 국소 미분의 곱
- 덧셈 노드 : dout 을 그대로 양쪽에
- 곱셈 노드 : dout × 상대편 입력 (서로 바꿔 곱함)
- 계층 = forward / backward 한 쌍

노트북 실습

1. 문제 3-1 MulLayer forward/backward
2. 문제 3-2 AddLayer
3. 문제 3-3 계층으로 사과 · 귤 그래프 조립

▶ 애니메이션 D3-3 연쇄 법칙 · 덧셈 / 곱셈 노드

핵심 개념

연쇄 법칙

```
z = t²,   t = x + y,   x = 3, y = 1
∂z/∂x = 2t · 1 = 8,   ∂z/∂y = 8
```

덧셈 노드

```
dx = dout * 1;   dy = dout * 1
```

곱셈 노드

```
dx = dout * y;   dy = dout * x
```

MulLayer

```
forward:  self.x, self.y = x, y; return x * y
backward: return dout * self.y, dout * self.x
```

체크포인트

곱셈 노드는 forward 입력을 저장해야 backward 가 가능하다 → 4 교시 " 계층은 값을 저장한다 ".

4 활성화 계층 — Relu · Sigmoid

13:00–13:50

학습 목표

- 계층은 forward 때 backward 에 필요한 값을 저장
- Relu: mask = (X <= 0) 저장 , mask 자리 기울기 0
- Sigmoid: 출력 out 저장 , dout * out * (1 − out)
- x = 0 이면 ReLU 기울기는 0 으로 정한다

노트북 실습

1. 문제 4-1 Relu
2. 문제 4-2 Sigmoid
3. 문제 4-3 Sigmoid backward 수치 미분 확인

▶ 애니메이션 D3-4 ReLU · Sigmoid 계층

핵심 개념

Relu

```
self.mask = (X <= 0); out[self.mask] = 0
backward: dout[self.mask] = 0
```

Relu 예

```
X = [[1.0, -0.5], [-2.0, 3.0]]
out = [[1, 0], [0, 3]]
backward(ones) = [[1, 0], [0, 1]]
```

Sigmoid

```
self.out = 1 / (1 + np.exp(-x))
dx = dout * self.out * (1.0 - self.out)
```

Sigmoid 예

```
x = [1.0, -2.0] → y = [0.7311, 0.1192]
y(1 − y) = [0.1966, 0.1050], 최대 0.25
```

체크포인트

Day 2 의 sigmoid_grad s(1 − s) 가 그대로 나온다 — 다만 x 로 다시 계산하지 않고 저장한 out 을 쓴다 .

5 Affine · Softmax-with-Loss

14:00–14:50

학습 목표

- Affine 계층 : Y = XW + b
- backward 3 식은 shape 를 맞추면 하나로 정해진다
- db = dout 을 행 방향 (axis=0) 으로 합
- SoftmaxWithLoss backward = (Y − T) / N

노트북 실습

1. 문제 5-1 Affine
2. 문제 5-2 softmax · cross_entropy_error
3. 문제 5-3 SoftmaxWithLoss
4. 문제 5-4 Day 2 로지스틱 기울기와 비교

▶ 애니메이션 D3-5 shape 퍼즐 · D3-6 y − t

핵심 개념

Affine backward

```
dX = dout @ W.T        # (N,M)@(M,D) = (N,D)
dW = X.T @ dout        # (D,N)@(N,M) = (D,M)
db = dout.sum(axis=0) # (M,)
```

MNIST 1 층

```
X (100,784) · W1 (784,100) · dout (100,100)
dW1 = X.T @ dout → (784, 100)
```

Softmax WithLoss

```
dx = (self.y - self.t) / N
x = [[0.3, 2.9, 4.0]], t = [2] → loss 0.3057
backward = [0.0182, 0.2452, -0.2634]
```

Day 2 연결

```
dout = (Y − T) / N  →  dW = X.T @ dout
= Day 2 의 dW = Xᵀ(Y − T)/N 과 같은 식
```

체크포인트

배치 크기와 은닉 크기가 둘 다 100 이면 틀린 식도 shape 가 맞는다 → 식은 shape 의 의미로 고른다 .

6 Gradient check

15:00–15:50

학습 목표

- TwoLayerNet: OrderedDict(Affine1, Relu1, Affine2) + SoftmaxWithLoss
- gradient() = forward → backward 를 역순으로
- rel_error < 1e-7 통과 · ~1e-4 의심 · > 1e-2 버그
- log 에 1e-12 (1e-7 이면 차이 ≈ 5e-7)

노트북 실습

1. 문제 6-1 TwoLayerNet.gradient
2. 문제 6-2 rel_error
3. 문제 6-3 gradient check 통과
4. 문제 6-4 버그 찾기

▶ 애니메이션 D3-7 Gradient check

핵심 개념

gradient

```
self.loss(X, t); dout = self.lastLayer.backward(1)
for layer in reversed(list(self.layers.values())):
    dout = layer.backward(dout)
```

rel_error

```
‖a − b‖ / (‖a‖ + ‖b‖ + 1e-12)
784-10-10: W1 2.1e-10 · b2 3.9e-10 → 통과
```

버그 ①

```
db = dout.mean(axis=0)  → b2 상대 오차 0.5
```

버그 ②

```
dX = dout @ W  (전치 빠짐) → W1 0.63, b1 0.56
W2 가 (10, 10) 이라 shape 오류 없이 실행
```

체크포인트

shape 가 맞아도 값이 틀릴 수 있다 → 새 계층을 만들 때마다 gradient check.

7 미니배치 SGD — 나선 분류

16:00–16:50

학습 목표

- epoch · batch · iteration
- iters_per_epoch = N // batch_size
- 매 epoch 셔플 : rng.permutation(N)
- train / test 정확도로 일반화 확인

노트북 실습

1. 문제 7-1 iteration 수 계산
2. 문제 7-2 미니배치 학습 루프
3. 문제 7-3 결정 경계 그리기

▶ 애니메이션 D3-8 미니배치 SGD — 나선 분류

핵심 개념

iteration

```
240 // 30 = 8 / epoch × 300 epoch = 2,400
MNIST: 60,000 // 100 = 600 / epoch
```

학습 루프

```
idx = rng.permutation(N)
for i in range(iters_per_epoch):
    b = idx[i*bs:(i+1)*bs]; g = net.gradient(X[b], t[b])
    for k in net.params: net.params[k] -= lr * g[k]
```

설정

```
TwoLayerNet(2, 100, 3) · batch 30 · lr 1.0
```

결과

```
epoch 1: test 0.35 → 50: 0.82 → 300: 0.983
train loss 1.097 → 0.039
```

체크포인트

결정 경계 스냅샷 epoch 1 / 10 / 50 / 300 — 은닉층 100 개가 직선들을 이어 붙여 나선을 감싼다 .

8 미니 프로젝트 — MNIST 2 층 MLP

17:00–17:50

강의 노트북 8 교시 · 784-100-10, 정확도 95% 이상

```
X = x.reshape(-1, 784).astype(np.float64) / 255.0
net = TwoLayerNet(784, 100, 10)  # 첫 손실 ≈ 2.3026
```

과제	내용	배점
1	전처리 — reshape · /255 · 레이블 정수 그대로	20
2	네트워크 생성 · 첫 손실 (≈ ln 10)	20
3	학습 루프 — test ≥ 0.95	30
4	학습 곡선 (손실 · 정확도)	20
5	결과 해석 (강사 채점)	10

채점 셀 실행 → 자동 채점 90 점 + 해석 10 점

▶ D3-9 에서 학습 과정 · 오답 예시를 먼저 보고 코드로 확인

오늘 만든 계층이 모두 들어간다

Affine	`dW = X.T @ dout`	5 교시
Relu	`mask` 자리 기울기 `0`	4 교시
SoftmaxWithLoss	`(Y - T) / N`	5 교시
gradient()	`backward` 를 역순으로	6 교시
미니배치 SGD	`batch 100 · lr 0.1`	7 교시

목표 · 참고 결과

test ≥ 0.95 (5 epoch 에 0.9531 도달)
10 epoch(6,000 iter) → train 0.9747 / test 0.9694

오늘의 핵심 정리

각 항목 옆 번호는 다시 볼 애니메이션

수치 미분 한계	step 당 손실 2P 번 — 79,510 개 → 159,020 번 , 역전파는 순전파 2~3 배	D3-1
계산 그래프	국소 계산 — 순방향은 값 , 역방향은 기울기 (사과 가격 기울기 2.2)	D3-2
연쇄 법칙	덧셈 노드는 그대로 , 곱셈 노드는 상대편 입력을 곱한다	D3-3
활성화 계층	Relu: mask 자리 0 · Sigmoid: dout * out * (1 − out)	D3-4
Affine	dX = dout @ W.T · dW = X.T @ dout · db = dout.sum(axis=0)	D3-5
SoftmaxWithLoss	(Y − T) / N — Day 2 의 "(예측 − 정답)" 이 다시 나온다	D3-6
Gradient check	rel_error < 1e-7 통과 — shape 가 맞아도 값은 틀릴 수 있다	D3-7
미니배치 SGD	epoch · batch · iteration → MNIST test 0.9694	D3-8 · D3-9

Day 4 로 가는 다리 — 학습을 잘 시키는 기술

오늘은 " 학습이 된다 " 까지 . 내일은 " 더 빠르고 안정적으로 , 덜 외우게 " 학습시킵니다

Day 3 (오늘)		Day 4 (내일)
`W -= lr * grad  (SGD)`	→	Momentum · AdaGrad · RMSProp · Adam
`weight_init_std = 0.01`	→	Xavier · He 초기화
`층마다 달라지는 활성값 분포`	→	Batch Norm
`train 0.9747 vs test 0.9694`	→	Dropout · weight decay (과적합 억제)
`lr 0.1 · batch 100 을 손으로`	→	하이퍼파라미터 탐색
`OrderedDict 로 쌓은 TwoLayerNet`	→	Sequential · Trainer 미니 프레임워크 (mynn.py)

SGD 의 약점 → 최적화 기법 → 초기화 → 정규화 → 하이퍼파라미터 → 미니 프레임워크

제출 : 강의 노트북 (MNIST 채점 셀 결과 포함) · 워크시트 오답 정리 캡처